

Implementing Zero Trust Security in AI-Powered Digital Voting System



Submitted by:

Hadia Moosa 2022-CS-119

Momina Rashad 2022-CS-137

Supervisor's Name

Sir Waqas Ali

Department of Computer Science

University of Engineering and Technology, Lahore

Contents

1	Background	8
1.1	Cybersecurity Challenges	8
1.2	Fraud Prevention	8
1.3	Ensuring Transparent Elections	8
2	Problem Statement	10
2.1	Data Breaches	10
2.2	Duplicate Voting	10
2.3	Tampered Vote Storage	10
2.4	Unauthorized Access	11
2.5	Lack of Transparency & Accountability	11
3	Objectives	12
3.1	Develop a Secure Digital Voting System Using Zero Trust Principles and AI-Based Fraud Detection	12
3.1.1	Zero Trust Security Model:	12
3.1.2	Continuous Authentication:	12
3.1.3	AI-Based Fraud Detection:	12
3.1.4	Anomaly Detection:	13
3.2	Implement Multi-Layered Authentication and Blockchain Technology to Secure the Vote and Storage	13
3.2.1	Multi-Layered Authentication:	13
3.2.2	Blockchain for Vote Storage:	13
3.2.3	Distributed Ledger:	13
3.2.4	Secure Vote Access:	13
3.3	Detect and Prevent Vote Tampering, Duplicate Voting, and Unauthorized Access in Real-Time	14
3.3.1	Vote Tampering Prevention:	14
3.3.2	Duplicate Voting Detection:	14
3.3.3	Real Time Fraud Detection:	14
3.3.4	Unauthorized Access Prevention:	14
4	Scope of the Project	15
4.1	System Coverage: User Authentication, Vote Casting, Vote Storage, and Fraud Detection	15

4.1.1	User Authentication:	15
4.1.2	Biometric Verification:	15
4.1.3	OTP and Password:	15
4.1.4	Vote Casting:	15
4.1.5	Vote Storage:	16
4.1.6	Fraud Detection:	16
4.1.7	LSTM based Fraud Detection:	16
4.1.8	Real Time Anomaly Detection:	16
4.2	Only Verified and Authenticated Users Can Cast Votes, Ensuring the Integrity of the Voting Process	16
4.3	Use of Blockchain for Secure Storage and AI for Monitoring Anomalies and Fraudulent Activities	17
4.4	Blockchain Technology:	17
4.4.1	Transparency:	17
4.4.2	AI for Monitoring Fraud:	17
4.5	Proposed Solution	17
4.5.1	Project Idea	17
4.5.2	Key Features of the Proposed System	18
5	Methodology	21
5.1	Introduction	21
5.2	Literature Review	21
5.3	Overview of Present Solutions	22
5.3.1	Traditional Voting Systems and Flaws	22
5.3.2	Existing AI Solutions for Fraud Detection	22
5.3.3	Zero Trust Security Models	22
5.4	Research & Related Work	22
5.4.1	Integrating AI and Blockchain for Digital Security	22
5.5	Earlier Solutions for Securing Electronic Voting	23
5.5.1	Blockchain for Transparency	23
5.6	Existing Methods for Digital Voting Security	23
5.6.1	Biometric Authentication	23
5.6.2	Blockchain Voting	23
5.6.3	Multi-Factor Authentication (MFA)	23
5.7	Gaps in Existing Solutions Identified	23
5.7.1	Insufficient AI for Real-Time Fraud Detection	23
5.7.2	Incomplete Adoption of Zero Trust Policies	24
5.7.3	Ongoing Vulnerabilities to Vote Tampering and Data Breaches	24
5.8	Tools, Technologies, and Frameworks	24
5.8.1	Backend Development	24
5.8.2	Frontend Development	24
5.8.3	AI Models for Authentication and Fraud Detection	25
5.8.4	Blockchain Technology	25
5.8.5	Security Protocols and Access Management	25
5.9	System Architecture	25

5.9.1	User Authentication and Registration	25
5.9.2	Secure Vote Casting System	25
5.9.3	Blockchain-Based Vote Storage	26
5.9.4	Fraud Detection and Anomaly Monitoring	26
5.9.5	Real-Time Security Monitoring and Identity Management . . .	26
6	Security threats and Zero Trust solutions applied	27
6.1	Unauthorized Access Threat	27
6.1.1	Threat	27
6.1.2	Zero Trust Principle Applied: “Verify Explicitly”	27
6.1.3	Solutions Implemented	27
6.1.4	Additional Detail	27
6.2	The Threat of Session Hijacking	28
6.2.1	Threat	28
6.2.2	Zero Trust Principle Applied: “Assume Breach”	28
6.2.3	Solutions Implemented	28
6.2.4	Additional Detail	28
6.3	Email OTP Spoofing or Replay Attack	28
6.3.1	Threat	28
6.3.2	Zero Trust Principle Applied: “Enforce Least Privilege”	28
6.3.3	Solutions Implemented	28
6.3.4	Additional Detail	29
6.4	Attempts at Duplicate Voting	29
6.4.1	Threat	29
6.4.2	Zero Trust Principle Applied: “Verify Explicitly”	29
6.4.3	Solutions Implemented	29
6.4.4	Additional Detail	29
6.5	Spoofing through Face Recognition	29
6.5.1	Threat	29
6.5.2	Zero Trust Principle Applied: “Assume Breach”	29
6.5.3	Solutions Implemented	30
6.5.4	Additional Detail	30
6.6	Blockchain Threat of Integrity Compromise	30
6.6.1	Threat	30
6.6.2	Zero Trust Principle Applied: “Verify Integrity Continuously”	30
6.6.3	Solutions Implemented	30
6.6.4	Additional Detail	30
6.7	Admin Role Misuse Threat	31
6.7.1	Threat	31
6.7.2	Zero Trust Principle Applied: “Enforce Least Privilege Access”	31
6.7.3	Solutions Implemented	31
6.7.4	Additional Detail	31

7	Implementation details, including Zero Trust measures and AI system features	32
7.1	Setting Up and Configuring the Flask Server	32
7.1.1	Flask Server Initialization	32
7.1.2	CORS Configuration	32
7.1.3	Development and Production Setup	32
7.1.4	Secure Handling of Environment Variables	32
7.2	Configuring MySQL Database	32
7.2.1	Database Connection	32
7.2.2	Primary Tables	33
7.3	Multi-Factor Authentication (MFA) via One-Time Passwords (Zero Trust Policy)	33
7.3.1	Overview of OTP Generation	33
7.3.2	Email Sending Process	33
7.3.3	Database Record for OTPs	33
7.3.4	User Verification Requirement	34
7.4	Voting on the Blockchain and Contract Deployment Using Web3.py .	34
7.4.1	Contract Compilation	34
7.4.2	Blockchain Deployment	34
7.4.3	Using Web3.py for Interaction	34
7.4.4	Ensuring Privacy and Transparency	34
7.5	Local Python-based Custom Blockchain	34
7.5.1	Integration with Flask Server	34
7.5.2	Blockchain Features	35
7.5.3	Blockchain APIs	35
7.5.4	Purpose of Local Blockchain	35
7.6	Zero Trust Architecture (Hardcore Security Focus)	35
7.6.1	Zero Trust Concept	35
7.6.2	Session Token Management	35
7.6.3	Security Against Persistent Sessions	36
7.6.4	Biometric Verification Enforcement	36
7.7	Face Verification AI System (Local Integration)	36
7.7.1	Overview	36
7.7.2	Registration Phase	36
7.7.3	Login Phase	36
7.7.4	Voting Phase	37
7.7.5	Security Measures for Face Verification	37
7.8	Full Voting Process (With Layered Protection)	37
7.8.1	Vote Flow	37
7.8.2	Duplicate Voting Protection	38
7.9	Role-Based Access Control (RBAC)	38
7.9.1	RBAC Integration	38
7.9.2	Role Privileges	38
7.9.3	API Access Verification	38
7.10	Error Handling, Monitoring, and Debugging	38

7.10.1	Error Handling	38
7.10.2	Debugging and Monitoring	39
7.10.3	User Feedback	39
8	Tests results and Evaluation	40
8.1	Send OTP Test	40
8.1.1	Objective:	40
8.1.2	Description:	40
8.1.3	Checks Performed:	40
8.1.4	Security Relevance:	40
8.2	Verify OTP Test	41
8.2.1	Objective:	41
8.2.2	Description:	41
8.2.3	Checks Performed:	41
8.2.4	Security Relevance:	42
8.3	Login Testing	42
8.3.1	Objective:	42
8.3.2	Description:	42
8.3.3	Security Relevance:	43
8.4	Protected Routes Testing	43
8.4.1	Objective:	43
8.4.2	Description:	43
8.4.3	Checks Performed:	44
8.4.4	Security Relevance:	44
9	Frontend Overview	45
10	Database Overview	52
11	Recommendations for Real-World Deployment	54
11.1	Secure Handling of Sensitive Information	54
11.2	Enhance Session Management and Authentication	54
11.3	Enhance Email OTP Delivery Mechanism	55
11.4	Enhance Face Recognition Security	55
11.5	Streamline Blockchain Interaction	55
11.6	Hardening MySQL Database	56
11.7	Best Practices for Smart Contract Security	56
11.8	Sophisticated Voting Fraud Detection	56
11.9	System Logging and Error Monitoring	57
11.10	Production-Grade Server Deployment	57
11.11	Authentication Enhancements	57
11.12	Legal Compliance and Data Privacy	58
11.13	Frontend Enhancements and User Experience	58

12 Conclusion	59
12.1 Key Takeaways from the Proposed Solution	59
12.1.1 Zero Trust Security Implementation:	59
12.1.2 AI-Powered Fraud Detection:	59
12.1.3 Blockchain-Based Vote Storage:	59
12.1.4 Enhanced Election Security:	59
12.2 Impact and Future Potential	60
12.2.1 Secure Identity Verification	60
12.2.2 Fraud-Resistant Financial Transactions	60
12.2.3 Tamper-Proof Record-Keeping	60

List of Figures

5.1	System Architecture of Secure Voting Platform	26
8.1	Testing Send OTP	41
8.2	Verify OTP	42
8.3	Login Testing	43
8.4	Protected Routes Testing	44
9.1	FrontPage Overview	45
9.2	FrontPage Overview	45
9.3	FrontPage Overview	46
9.4	About	46
9.5	SignUp Form	47
9.6	Login Form	47
9.7	Candidates to Vote Page Overview	48
9.8	RBAC Page Overview	48
9.9	Election Results	49
9.10	Validating Blockchain	49
9.11	Fraud Detection Page Overview	50
9.12	Contact Form	50
9.13	Mail received	51
9.14	Fraud Detection Page Overview	51
10.1	Database Tables	52
10.2	Session Management	53
10.3	Authenticated users are restricted to a single vote	53
10.4	Encoding of Facial Data	53

Chapter 1

Background

In the last few years, electronic voting systems have emerged as a promising alternative to traditional paper-based voting, offering convenience, speed, and accessibility. However, with all such advancements come significant security concerns that compromise the integrity of the voting process. These weaknesses have raised a growing concern over the credibility and fairness of elections in the digital age.

1.1 Cybersecurity Challenges

Electronic voting systems are vulnerable to a large number of cyber threats, such as hacking, phishing, and denial-of-service attacks. These threats are generally aimed either at manipulating votes or bringing the voting process to a halt altogether. More specifically, cyber attacks may aim at central servers, infect voter databases, or intercept votes during transmission over the internet, which can lead to data breaches and vote manipulation.

1.2 Fraud Prevention

One of the main concerns in electronic voting is being able to confirm that each person can only vote once. In traditional systems, human oversight and physical presence at voting stations are a guarantee against multiple voting. Digital systems, however, are vulnerable to manipulation, for example, by using hijacked identities, fake voter registration, and unauthorized multiple casting of votes. This opens up the possibility of a person casting more than one vote under different identities, which can alter the outcome of elections.

1.3 Ensuring Transparent Elections

For electronic voting to become widely accepted and trusted, it must be transparent and verifiable. Vote tampering—vote tallies manipulated or records changed—is a grave threat to democratic process. Without constraints to check that votes are

authentic and of integrity, the entire election process would be open to question and trigger public distrust and potential legal action.

Since such threats have been rising, it is increasingly important to impose advanced security systems to secure electronic voting systems. Zero Trust Security (ZTS) concepts and Artificial Intelligence (AI) solutions can play a critical role in maintaining the integrity of digital elections. By authenticating and securing every element of the voting system—from voter verification to vote storage—such technologies can guard against fraud, unauthorized access, and tampering.

Here, the amalgamation of Zero Trust Security and AI-driven fraud detection presents a robust solution for digital voting system security. Zero Trust Security, which assumes the default position that nobody is trustworthy, employs strict verification at every level. While AI may keep scanning voting activity, detect anomalies, and alert fraudulent activity in real time.

Chapter 2

Problem Statement

Although there has been improvement in electronic voting systems, they are still vulnerable to various types of attacks and tampering, which degrade the integrity and security of the election process. Specifically, the main issues of current electronic voting systems are:

2.1 Data Breaches

Private data identification data, voting record, and personal data of voters are typically stored in central databases, which become the target of hackers. Data breaches can lead to the stolen data being utilized for theft, which can be used to render voter identities or vote counts unreliable. Such an attack also dissolves the faith in the election process that the public has.

2.2 Duplicate Voting

Duplicate voting is one of the largest problems in e-voting because it is conceivable that an individual can vote more than once, either by using false identities or system vulnerabilities. Such duplicate voting can occur when authentication techniques fail to operate as they are supposed to authenticate the voters' identity or when the system does not have adequate protection to identify multiple submissions by the same individual.

2.3 Tampered Vote Storage

Votes in computer systems are typically stored in central databases or routed over the internet, and hence susceptible to tampering. A hacker or insider can alter vote records, either while in transit or after casting, and manipulate votes. The integrity of stored votes is exposed to threats without proper safeguards.

2.4 Unauthorized Access

Most contemporary voting systems rely on traditional security practices such as passwords and static authentication that can easily be compromised through phishing attacks or stolen credentials. Once a malicious user has a voter's credentials or the election infrastructure, they can manipulate fake votes or tamper with the outcome of the election. In addition, improper insider access by election authorities may lead to manipulation of fake votes.

2.5 Lack of Transparency & Accountability

Another critical issue with existing systems is the lack of transparency and accountability in casting votes and storage. Without verification and tracking of the votes in real time, neither the voters nor the election bodies can ascertain whether the votes are being counted rightly or if the votes have been tampered with. This causes a lack of trust, such that the populace can question the legitimacy of the election result. These vulnerabilities make the integrity, security, and transparency of digital voting, and one that openly endangers the validity of elections. The task now is to develop a system that not only protects against such vulnerabilities but also provides real-time fraud detection, secure vote preservation, and total accountability during the process of voting. As an answer to such dilemmas, this project aims to develop an AI-driven digital voting system with the integration of Zero Trust Security concepts. The system will have next generation authentication methods, blockchain-safe storage of votes, and AI-powered fraud detection for addressing the issues highlighted above earlier, maintaining authenticity and safety of e-governance.

Chapter 3

Objectives

The main objective of this project is to create a secure AI-based digital election system that incorporates Zero Trust Security principles, AI-based fraud detection, and blockchain technology to verify the integrity, security, and transparency of the electoral process. Below is a detailed explanation of the project objectives:

3.1 Develop a Secure Digital Voting System Using Zero Trust Principles and AI-Based Fraud Detection

3.1.1 Zero Trust Security Model:

This objective entails the implementation of a Zero Trust model, which assumes that every request for access, both internal and external, is at risk. Therefore, all requests to access the voting system (both from election officials and voters) will always be and be thoroughly checked. This will ensure only authorized users and activities are permitted at any given time.

3.1.2 Continuous Authentication:

Authentication will not be done only once. Users will always be under observation and re-authenticated, especially for significant actions like casting votes and viewing sensitive information.

3.1.3 AI-Based Fraud Detection:

The platform will utilize AI algorithms to detect anomalous patterns of voting or unusual behaviors. For example, the platform will look into voters' trends to determine unusual activity such as voting repeatedly using the same voter or unusual login places

3.1.4 Anomaly Detection:

Real-time AI driven anomaly detection will track voting patterns to detect and prevent any suspicious activity from being carried out in real-time before they affect the integrity of the election.

3.2 Implement Multi-Layered Authentication and Blockchain Technology to Secure the Vote and Storage

3.2.1 Multi-Layered Authentication:

The project will implement multi-factor authentication (MFA) to ensure that each voter is securely verified before they can cast their vote. This will include:

- **Biometric Verification:** A face recognition system (using technologies like FaceNet or similar AI models) to confirm the identity of voters.
- **OTP (One-Time Password):** Voters will be sent a unique, time-sensitive OTP to their registered phone or email as a secondary layer of verification.
- **Password:** A traditional password requirement to further enhance security.

By combining these layers, the system will prevent unauthorized access from both external hackers and internal users, ensuring that only verified individuals can participate in the voting process.

3.2.2 Blockchain for Vote Storage:

For data integrity, the project will utilize blockchain technology for tamper-proof vote storage. This will render it impossible to alter votes after they have been cast, as blockchain records are immutable and cannot be altered.

3.2.3 Distributed Ledger:

All votes will be stored as a transaction on a distributed ledger (e.g., Hyperledger Fabric or Ethereum) in a way that makes votes transparent, auditable, and irreversibly registered in a way that guarantees authenticity.

3.2.4 Secure Vote Access:

Blockchain will also help achieve role-based access control (RBAC), where only allowed election officials can see part of the information or make changes to the system. The use of blockchain prevents unauthorized individuals from manipulating or deleting vote records.

3.3 Detect and Prevent Vote Tampering, Duplicate Voting, and Unauthorized Access in Real-Time

3.3.1 Vote Tampering Prevention:

The system will include blockchain-based voting that encrypts every vote and makes it impossible for it to be altered once it is submitted. With blockchain, even the system administrators will not be in a position to tamper with the vote records, upholding the integrity of the election process.

3.3.2 Duplicate Voting Detection:

Fraud prevention will be facilitated by AI. It will keep track of voting patterns in real time to identify duplicate voting. For example, if the same voter attempts to vote multiple times under different accounts or various locations, the system will immediately flag this as suspicious, preventing the fraudulent attempt.

3.3.3 Real Time Fraud Detection:

Using anomaly detection models such as LSTM (Long Short-Term Memory), the system will be constantly scanning voting activities for patterns of fraud. These will range from suspicious patterns of voting, multiple voting from a single IP address, or voting after or prior to the set times.

3.3.4 Unauthorized Access Prevention:

The system will implement Zero Trust policies so that unauthorized users (including potential insiders) are not granted access to the system. All activity, whether it's whether it is an administrator updating or a voter casting a vote, will be under constant monitoring. Even after a voter has authenticated once, their actions will be periodically verified to prevent any unauthorized attempt to alter the system.

Chapter 4

Scope of the Project

The scope of this project outlines the specific areas that will be addressed by the system and the overall boundaries of the work. Below is a detailed breakdown of the project scope:

4.1 System Coverage: User Authentication, Vote Casting, Vote Storage, and Fraud Detection

4.1.1 User Authentication:

The system will ensure that only verified individuals are allowed to participate in the voting process. Using multi layered authentication (including biometric verification, OTP, and passwords), the system will ensure that every voter is authenticated before they are permitted to cast their vote.

4.1.2 Biometric Verification:

The use of FaceNet or similar AI models for face recognition ensures that voters cannot impersonate others. This is particularly important in preventing identity theft and impersonation during the voting process.

4.1.3 OTP and Password:

These additional layers of verification add redundancy and further security to ensure that only the legitimate voter can access the system.

4.1.4 Vote Casting:

Once authenticated, voters will cast their votes through a secure web portal or mobile app. AI will monitor the process to check for duplicate voting or any other forms of fraudulent attempts.

The system will provide a Just-in-Time (JIT) access mechanism to ensure that voters only have access to the voting system for a limited time, preventing vote manipulation.

4.1.5 Vote Storage:

After the vote is cast, it will be stored on a blockchain ledger, which guarantees that the vote is immutable and secure. This approach prevents the possibility of vote tampering or unauthorized deletion. Blockchain also guarantees that the vote records are transparent and auditable, providing full accountability.

4.1.6 Fraud Detection:

AI will play a key role in fraud detection by using anomaly detection algorithms to monitor unusual voting patterns. The system will also check for fake user registrations and flag any inconsistencies during the voting process.

4.1.7 LSTM based Fraud Detection:

The AI system will continuously monitor voting behavior and flag suspicious activity in real time, such as multiple votes from the same IP address or inconsistent voting times.

4.1.8 Real Time Anomaly Detection:

The system will also include real-time alerts if it detects any abnormal behavior, such as voting outside of allowed hours or repeated login attempts from unrecognized devices or locations.

4.2 Only Verified and Authenticated Users Can Cast Votes, Ensuring the Integrity of the Voting Process

To ensure the integrity of the voting process, only those users who pass the authentication checks will be allowed to cast their vote. This guarantees that only legitimate voters can participate in the election, preventing fraud and manipulation.

By enforcing this authentication mechanism, the system prevents unauthorized users, including impersonators and hackers, from voting.

4.3 Use of Blockchain for Secure Storage and AI for Monitoring Anomalies and Fraudulent Activities

4.4 Blockchain Technology:

The system will leverage blockchain to store votes securely. This technology ensures that votes are immutable and cannot be tampered with, ensuring transparency and security.

4.4.1 Transparency:

Blockchain allows for full auditability, meaning that election authorities and other stakeholders can review the votes and verify their authenticity at any time.

4.4.2 AI for Monitoring Fraud:

The system will incorporate AI models to detect fraud in real-time. Through continuous monitoring, AI will flag any suspicious voting patterns or anomalies, ensuring that fraudulent behavior is detected and prevented before it can alter the outcome of the election.

4.5 Proposed Solution

With the increasing risks of cyber threats, unauthorized access, and vote tampering in digital elections, this project proposes a multi-layered AI-powered digital voting system that operates under **Zero Trust Security** principles. This approach ensures that every action within the voting system is continuously verified and secured, from voter authentication to vote storage and fraud detection. By integrating **Multi-Factor Authentication (MFA)**, **Blockchain Technology**, and **AI-based Fraud Detection**, the system provides a secure, transparent, and tamper-proof voting environment that can be applied to real-world elections.

4.5.1 Project Idea

The core idea behind this project is to develop a highly secure and reliable AI-powered digital voting system that incorporates Zero Trust Security principles to eliminate fraud, unauthorized access, and data breaches in online elections. Unlike traditional voting systems that operate on a perimeter based security model, this system follows a **Zero Trust framework**, where:

- No user or device is inherently trusted, even if they are within the system.

- Every authentication attempt is verified continuously using AI-driven biometric analysis and multi-layered security checks.
- Votes are securely stored in an immutable blockchain ledger, ensuring that they cannot be altered, manipulated, or deleted.
- AI-powered fraud detection mechanisms operate in real-time to detect and prevent duplicate votes, fake registrations, and suspicious voting behaviors.

This approach ensures that only verified voters can cast their votes, and all votes remain secure, anonymous, and immutable, significantly reducing the risks of vote manipulation or cyber-attacks.

4.5.2 Key Features of the Proposed System

To ensure the highest level of security, transparency, and fraud prevention, the proposed AI-powered digital voting system will include the following key features:

Multi-Factor Authentication (MFA) for Secure Voter Verification

Authentication is one of the most critical aspects of securing a digital voting system. This project integrates **Multi-Factor Authentication (MFA)** to ensure that only legitimate, verified users can access the system and cast votes. The MFA system will include:

- **Biometric Authentication:**
 - Face Recognition (FaceNet API) or Government ID Verification to authenticate voters.
 - Prevents impersonation or unauthorized access attempts.
- **One-Time Password (OTP) Authentication:**
 - An OTP is sent to the voter's registered mobile number or email for an additional layer of verification.
- **Password-Based Security:**
 - Users must create a strong password during registration, which is used as part of the login process.
- **Zero Trust Implementation:** Every login attempt undergoes continuous verification, meaning that even after initial authentication, the system will keep checking for any suspicious activity.

Blockchain-Based Vote Storage for Immutable Security

To eliminate the possibility of vote tampering, unauthorized access, or deletion, this system will store all votes on a **blockchain ledger**, ensuring:

- **Tamper-Proof Storage:** Once a vote is recorded, it cannot be modified or deleted.
- **Decentralized & Transparent:** The blockchain ledger ensures that votes are publicly verifiable but remain anonymous, enhancing transparency.
- **Secure Cryptographic Encryption:** Votes are encrypted before being stored, ensuring that no third party (including system administrators) can modify them.
- **Blockchain Technology Used:** Hyperledger Fabric or Ethereum Smart Contracts for secure, decentralized vote recording.
- **Role-Based Access Control (RBAC):** Ensures that election officials can only view vote summaries without accessing individual votes.
- **Zero Trust Implementation:** Just-in-Time (JIT) Access ensures that even system administrators have time-limited, strictly monitored access to voting data.

AI-Powered Fraud Detection to Prevent Election Manipulation

To detect and prevent vote duplication, unauthorized access, or fake voter registrations, the system will integrate **AI-driven fraud detection mechanisms**:

- **Duplicate Vote Detection:**
 - AI-based Computer Vision models (CNNs) will compare voter identities to ensure that no individual casts multiple votes using different credentials.
- **Fake Voter Registration Detection:**
 - AI-powered Natural Language Processing (NLP) models will analyze voter registration data to identify fake accounts or suspicious voter applications.
- **Machine Learning-Based Vote Manipulation Detection:**
 - AI will analyze voting patterns and flag unusual activities, such as mass voting from the same IP address, abnormal login times, or improbable voting behaviors.
- **Zero Trust Implementation:** Continuous AI-driven security monitoring ensures that all interactions are constantly verified for authenticity and legitimacy.

Real-Time Anomaly Monitoring & AI Threat Detection

The system will incorporate real-time anomaly monitoring to track and flag suspicious voting behaviors:

- **Unusual Login Locations:** AI will monitor whether voters are accessing the system from unexpected geographical locations.
- **Multiple Voting Attempts:** AI will detect users attempting to log in from different devices with different credentials.
- **Unauthorized Admin Access:** AI will monitor system administrators and election officials for any unauthorized or unusual activities, ensuring that no insider threats manipulate votes.
- **AI Models Used for Threat Detection:**
 - LSTM-Based Anomaly Detection to identify suspicious voting behaviors.
 - PyOD (Python Outlier Detection) for detecting fraudulent activity.
 - GPT-3/NLP Models to scan voter registration databases for fake profiles or bot-generated voter accounts.
- **Zero Trust Implementation:** AI continuously monitors voting behavior and login attempts to detect anomalies before they become threats.

Chapter 5

Methodology

5.1 Introduction

The methodology outlines the process of designing, developing, testing, and securing the AI-based e-voting system based on Zero Trust Security principles. It includes the selection of tools, technologies, security frameworks, and testing procedures.

The research process consists of five main stages:

1. **Literature Review** – Investigating AI security, Zero Trust models, and blockchain-based voting systems.
2. **Threat Analysis** – Identifying vulnerabilities in AI-enabled voting applications.
3. **Zero Trust Implementation** – Designing and implementing a security framework with continuous authentication and verification.
4. **Simulation & Testing** – Testing system security via attack simulations and penetration testing.
5. **Paper Writing & Recommendations** – Documenting findings, proposing improvements, and suggesting best practices.

5.2 Literature Review

This section evaluates existing research on securing digital voting through AI, Zero Trust security, and blockchain technology. It highlights challenges in current systems and supports the need for a more secure, fraud-resistant e-voting solution.

5.3 Overview of Present Solutions

5.3.1 Traditional Voting Systems and Flaws

Traditional paper-based voting systems have long been used but exhibit several critical vulnerabilities:

- **Human Errors:** Manual ballot counting can lead to mistakes and misinterpretations.
- **Corruption:** Risks of ballot stuffing, vote tampering, and voter intimidation are prevalent.
- **Cost and Inefficiency:** High logistical and resource requirements make large-scale elections challenging.

5.3.2 Existing AI Solutions for Fraud Detection

AI technologies have demonstrated potential in detecting fraud across digital systems:

- **Anomaly Detection Models:** Machine learning models like Random Forest, SVM, and Neural Networks identify abnormal voting patterns.
- **NLP for Profile Verification:** Natural Language Processing helps detect fake or automated voter registrations.
- **Biometric Authentication:** AI-powered biometric systems like FaceNet enhance user verification and prevent impersonation.

5.3.3 Zero Trust Security Models

Zero Trust Security (ZTS) principles strengthen digital platforms by:

- **Verifying Every Access Request:** Eliminating implicit trust and verifying all users and devices.
- **Implementing Continuous Authentication:** Enforcing ongoing identity verification through Multi-Factor Authentication (MFA).
- **Granular Access Control:** Restricting access based on user roles (RBAC) to protect sensitive operations.

5.4 Research & Related Work

5.4.1 Integrating AI and Blockchain for Digital Security

Recent research explores combining AI with blockchain to enhance digital security:

- **Blockchain with AI Monitoring:** AI detects anomalies in blockchain transactions and prevents unauthorized ledger modifications.
- **Smart Contracts with AI Oversight:** AI models oversee smart contract operations to detect fraud in blockchain-based voting systems.
- **Blockchain for Transparency:** Immutable records ensure votes are tamper-proof and verifiable.

5.5 Earlier Solutions for Securing Electronic Voting

5.5.1 Blockchain for Transparency

Blockchain technology ensures the integrity of votes by providing an immutable ledger, securing the transparency and trustworthiness of the election process.

5.6 Existing Methods for Digital Voting Security

5.6.1 Biometric Authentication

Biometric technologies, such as facial recognition and fingerprint scanning, enhance voter verification and mitigate identity theft.

5.6.2 Blockchain Voting

Blockchain records votes securely, ensuring they cannot be altered or deleted after casting.

5.6.3 Multi-Factor Authentication (MFA)

Combining multiple authentication methods strengthens system security, reducing unauthorized voting risks.

5.7 Gaps in Existing Solutions Identified

5.7.1 Insufficient AI for Real-Time Fraud Detection

Most current systems lack real-time AI-based monitoring for detecting voting anomalies as votes are cast.

Lack of Continuous Monitoring

Systems usually detect fraud only after elections end, missing the opportunity for immediate response to suspicious activities.

5.7.2 Incomplete Adoption of Zero Trust Policies

Many digital voting platforms still rely on static security measures and do not implement continuous verification strategies.

Absence of Continuous Authentication

Failure to maintain Zero Trust policies opens the door to unauthorized access and vote manipulation during elections.

5.7.3 Ongoing Vulnerabilities to Vote Tampering and Data Breaches

Digital voting platforms remain prone to:

- **Vote Tampering:** Lack of strong blockchain implementations allows post-casting manipulation.
- **Data Breaches:** Hackers can access voter information, leading to fraud and compromised elections.

These gaps highlight the critical need for an integrated solution using AI, blockchain, and Zero Trust security.

5.8 Tools, Technologies, and Frameworks

5.8.1 Backend Development

The backend development leverages secure frameworks and robust programming languages:

- **Flask/Django (Python):** API development, user authentication, vote processing, and AI model integration.
- **Stripe (Payment Integration):** Used to implement micro-donations or fees for registration/authentication.
- **SQLAlchemy (ORM):** Manages database models and ensures data integrity and security.

5.8.2 Frontend Development

For a dynamic, user-friendly interface:

- **React.js:** User-friendly interfaces for secure web and mobile voting.
- **Material UI:** Used for styling to create modern, responsive UI components.
- **Vite:** For fast development builds and efficient resource bundling.

5.8.3 AI Models for Authentication and Fraud Detection

AI algorithms and deep learning models detect anomalies and ensure security during voting:

- **FaceNet API (Hugging Face):** Facial recognition for voter authentication.
- **PyOD (Python Outlier Detection):** AI models for detecting fraudulent voting patterns.
- **LSTM Model (TensorFlow/Keras):** Deep learning for real-time anomaly detection during voting.

5.8.4 Blockchain Technology

To ensure transparency and immutability of votes:

- **Ethereum (Smart Contracts):** Enables secure, blockchain-based voting and record keeping.
- **Hyperledger Fabric:** A permissioned blockchain for secure, transparent voting.

5.8.5 Security Protocols and Access Management

To secure user identities and access controls:

- **JWT Authentication:** Token-based secure authentication.
- **OAuth Protocol:** Secure user authorization for system interactions.
- **Role-Based Access Control (RBAC):** Access restrictions for election officials and voters.

5.9 System Architecture

5.9.1 User Authentication and Registration

- **Biometric Verification:** FaceNet-based or government ID verification.
- **Zero Trust Implementation:** Continuous re-verification for all user activities.

5.9.2 Secure Vote Casting System

- **Encrypted Voting Platforms:** Secure web and mobile apps for casting votes.
- **AI-Based Fraud Detection:** Real-time monitoring of voting behavior to detect anomalies.

5.9.3 Blockchain-Based Vote Storage

- **Immutable Ledger:** Blockchain ensures secure, tamper-proof vote storage.

5.9.4 Fraud Detection and Anomaly Monitoring

- **LSTM Models:** Real-time detection of suspicious voting activities.
- **NLP for Registration Verification:** Identification of bot-generated or fake profiles.

5.9.5 Real-Time Security Monitoring and Identity Management

- **Continuous Monitoring:** AI tracks user behavior, IP addresses, and locations.
- **Zero Trust Identity Management:** Persistent monitoring for session-based security.

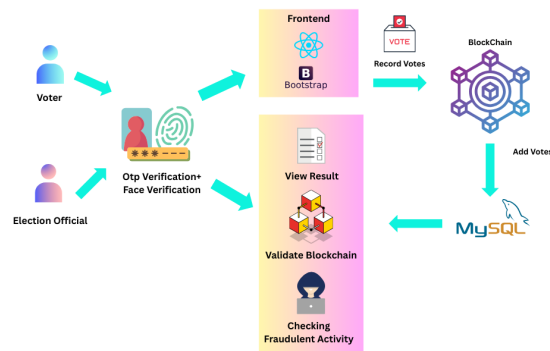


Figure 5.1: System Architecture of Secure Voting Platform

Chapter 6

Security threats and Zero Trust solutions applied

6.1 Unauthorized Access Threat

6.1.1 Threat

Criminals might circumvent standard login procedures by capturing credentials or session tokens.

6.1.2 Zero Trust Principle Applied: “Verify Explicitly”

Always require strong evidence of onboarding an entity and their identity.

6.1.3 Solutions Implemented

- **Face Authentication Login System:** The user is authenticated through live face detection and facial feature encoding, which is more secure than simple credential-based logins.
- **Token-based Session Management:** Tokens are randomly generated, unchanging (`secrets.token_hex(16)`) assigned at login, stored in a database, and validated at each protected route check.
- **Binding Tokens to IP Addresses:** Tokens are checked against the originating IP address; even if a token is captured, it cannot be used from another device thus mitigating risk.

6.1.4 Additional Detail

Should the user’s IP shift, or the token is used in other locations, the session is terminated in real-time.

6.2 The Threat of Session Hijacking

6.2.1 Threat

A malicious individual might intercept a session token and exploit it to act as a user.

6.2.2 Zero Trust Principle Applied: “Assume Breach”

Act as if the session is compromised and take necessary actions proactively.

6.2.3 Solutions Implemented

- **Scoped Access for Short-lived Sessions:** Sessions are designed to automatically expire within five minutes, severely mitigating the opportunity for a token to be exploited.
- **Rolling Expiry Mechanism:** Every action performed by the user (such as voting) is considered valid and refreshes the session expiry, ensuring that active users remain logged in securely while inactive sessions quickly time out.

6.2.4 Additional Detail

Only active requests within the `require_valid_session` decorator can refresh the expiry timestamp, which remains immutable for inactive sessions.

6.3 Email OTP Spoofing or Replay Attack

6.3.1 Threat

A fraudulent registration could take place if an attacker reused an OTP sent to another user.

6.3.2 Zero Trust Principle Applied: “Enforce Least Privilege”

Grant permission and access only after thorough verification checks.

6.3.3 Solutions Implemented

- **Short-Lived OTPs:** The attack window is minimal, as OTPs are valid for only 5 minutes.
- **Single-use OTP Mechanism:** New OTPs overwrite older ones stored in the database, preventing reuse.

- **Backend Validation on Every Request:** Since frontend validation is untrusted, the server directly verifies email and OTP combinations.

6.3.4 Additional Detail

Because the server expects an OTP that matches both expiry and the associated email ID, attackers are unable to reuse captured OTPs effectively.

6.4 Attempts at Duplicate Voting

6.4.1 Threat

A user may try to vote more than once by using different accounts or identities.

6.4.2 Zero Trust Principle Applied: “Verify Explicitly”

Ensure users are uniquely identifiable prior to important actions, such as casting a vote.

6.4.3 Solutions Implemented

- **Voting ID Hash Verification:** Each voting ID is hashed (SHA-256) and checked for duplicates without retaining raw identifiers; user privacy is safeguarded.
- **Face Duplication Prevention:** Face encodings submitted are matched against stored face encodings to prevent a single individual from voting multiple times under different identities.

6.4.4 Additional Detail

Checks for resemblance between face encodings (calculated using Euclidean distance) guarantee that even minor discrepancies between faces cannot bypass duplication detection.

6.5 Spoofing through Face Recognition

6.5.1 Threat

Photos or pre-recorded video footage can be used by malicious users to deceive the system.

6.5.2 Zero Trust Principle Applied: “Assume Breach”

All images are considered untrustworthy until verified through real-time validation.

6.5.3 Solutions Implemented

- **Requirement of a Single Face:** Requests containing no face or multiple faces are immediately blocked.
- **Capture in Real-Time:** The frontend requires the user to take a live photo directly from their device's camera prior to submission; images cannot be pre-uploaded.

6.5.4 Additional Detail

During the live image capture, the image is encoded in base64, which helps protect it from interception during transmission.

6.6 Blockchain Threat of Integrity Compromise

6.6.1 Threat

The possibility exists that voting records could be modified through alteration of blockchain data.

6.6.2 Zero Trust Principle Applied: “Verify Integrity Continuously”

Forensically ensure that data cannot be changed and is continuously verifiable.

6.6.3 Solutions Implemented

- **Proof of Work (PoW):** Adding new blocks requires solving a cryptographic challenge (generating a hash starting with four zeros), making casual alterations highly unlikely.
- **Hash Chain Linking:** Each block includes the hash of the previous block, so any tampering is immediately detectable.
- **Validation Endpoint:** Nodes or administrators can use the `/validate-chain` endpoint to verify blockchain integrity on demand.

6.6.4 Additional Detail

If even a single block is compromised, the disruption in hash continuity will immediately invalidate the blockchain.

6.7 Admin Role Misuse Threat

6.7.1 Threat

Improper users may attempt to impersonate administrators and modify election information.

6.7.2 Zero Trust Principle Applied: “Enforce Least Privilege Access”

Granular restrictions are required for all sensitive and critical actions.

6.7.3 Solutions Implemented

- **Hardcoded Secret for Admin Registration:** Only users entering a predefined secret base string (“momina”) can register as administrators.
- **Role-Based Access Control:** Users and officials are stored separately in distinct containers, preventing unauthorized role changes after registration.

6.7.4 Additional Detail

Admin privileges cannot be obtained via URL discovery or simple client-side code since backend enforces role validation.

Chapter 7

Implementation details, including Zero Trust measures and AI system features

7.1 Setting Up and Configuring the Flask Server

7.1.1 Flask Server Initialization

The backend server is created using Flask, initialized with a parameterized constructor: `Flask_Init(__name__)`.

7.1.2 CORS Configuration

CORS is configured using the `flask_cors` library to permit safe communication with frontend clients such as React applications.

7.1.3 Development and Production Setup

For development and testing, the server runs locally on `http://127.0.0.1:5000`. It is structured to be easily adapted for production deployments in the future.

7.1.4 Secure Handling of Environment Variables

SMTP email credentials, JWT secret keys, and Web3 provider URLs are loaded securely from environment variables to prevent hardcoded sensitive data.

7.2 Configuring MySQL Database

7.2.1 Database Connection

The system uses `mysql.connector` to establish a connection to a MySQL database named `akhiridb`.

7.2.2 Primary Tables

The following key tables are designed:

- **users:** Contains user information including email addresses, hashed passwords, and Base64 encoded face encodings.
- **votes:** Stores votes submitted by users along with their corresponding blockchain transaction hashes.
- **blocks:** Represents the locally managed custom blockchain. Each block includes an index, timestamp, proof, previous hash, and vote data.
- **sessions:** Maintains active session tokens with associated user IDs, IP addresses, and expiration times to manage authenticated sessions.
- **otps:** Temporarily stores email OTPs for two-factor authentication (2FA) purposes.
- **vote_logs:** Captures all user voting activities, whether successful, duplicate, or fraudulent, along with detailed metadata for audit purposes.

7.3 Multi-Factor Authentication (MFA) via One-Time Passwords (Zero Trust Policy)

7.3.1 Overview of OTP Generation

The system generates a 6-digit one-time password (OTP) for each authentication attempt. These OTPs are valid only once, ensuring additional security as part of the Zero Trust policy.

7.3.2 Email Sending Process

The process of sending OTPs is facilitated through SMTP Gmail servers:

- The communication is secured by TLS, with the session running on port 587.
- OTPs are sent via emails to users' registered email addresses.

7.3.3 Database Record for OTPs

The following details are stored in the database for each OTP:

- **Account email ID:** The email address associated with the OTP.
- **Creation Timestamp:** The exact time the OTP was created.
- **Deletion Timestamp:** OTPs are automatically deleted within 5 minutes of generation to ensure time-sensitive security.

7.3.4 User Verification Requirement

Only verified users, having successfully authenticated with their OTP, are granted the ability to proceed. This ensures that actions such as logging in or voting are only performed by authenticated users.

7.4 Voting on the Blockchain and Contract Deployment Using Web3.py

7.4.1 Contract Compilation

The voting system uses a Solidity contract, `VotingContract.sol`, which is compiled using `solcx`. During the compilation process, the Contract ABI (Application Binary Interface) and bytecode are generated.

7.4.2 Blockchain Deployment

The compiled contract is deployed on a local Ganache blockchain, which simulates an Ethereum environment for testing and deployment purposes.

7.4.3 Using Web3.py for Interaction

The `web3.py` library is used for interacting with the deployed contract. The following functionalities are provided:

- Votes are stored on-chain, ensuring immutability and transparency.
- The contract allows interactions such as calling functions and sending transactions.

7.4.4 Ensuring Privacy and Transparency

To protect voter privacy while maintaining transparency, each vote is hashed before being saved on the blockchain. This ensures that individual votes cannot be traced back to specific users, while still maintaining a transparent and immutable record.

7.5 Local Python-based Custom Blockchain

7.5.1 Integration with Flask Server

The backend server developed using Flask integrates a Python-based blockchain represented by a class named `Blockchain`.

7.5.2 Blockchain Features

- **Proof of Work:** Protects block creation by ensuring computational work is performed during mining.
- **Chained Hashes:** Each block links to the previous one using cryptographic hashes, ensuring tamper-evidence.
- **Vote Recording:** Votes are securely stored inside the blocks, ensuring immutability.

7.5.3 Blockchain APIs

- **/add-block:** Manual method for adding new blocks to the chain.
- **/validate-chain:** Endpoint to validate the entire blockchain to detect tampering.

7.5.4 Purpose of Local Blockchain

The local blockchain serves as a secondary independent verification mechanism. It provides double-layer security if the main Ethereum network fails or is under dispute.

7.6 Zero Trust Architecture (Hardcore Security Focus)

7.6.1 Zero Trust Concept

The Zero Trust model is based on the philosophy of “*Never trust, always verify*”. No user or system is automatically trusted, even after successful login.

7.6.2 Session Token Management

- **Re-confirmation of Session Tokens:** After every API call, tokens must be re-validated.
- **IP Address Binding:** Tokens are valid only if coming from the same IP used during login.
- **Short Token Lifespan:** Tokens expire after 5 minutes; a new token is issued if the user remains active.
- **Handling Expired or Misplaced Tokens:** Verification is mandatory on expiration or IP mismatch.

7.6.3 Security Against Persistent Sessions

Persistent long-lived sessions are not trusted. Every user action is subjected to security verification regardless of previous authentications.

7.6.4 Biometric Verification Enforcement

Biometric authentication via face recognition is mandatory during sensitive actions such as voting. This prevents:

- Session hijacking
- Unauthorized access
- Device theft exploitation

7.7 Face Verification AI System (Local Integration)

7.7.1 Overview

The system uses the `face_recognition` Python library to provide AI-based facial authentication locally without external dependencies.

7.7.2 Registration Phase

- The user uploads a holographic face image.
- A unique face encoding (digital facial fingerprint) is generated.
- The encoding is securely stored in the database for future authentication.

7.7.3 Login Phase

- The user provides a live facial image.
- A new face encoding is generated from the live image.
- The new encoding is compared to the stored encoding using a distance metric.
- If the encoding matches within the predefined threshold, the user is authenticated.

7.7.4 Voting Phase

During voting:

- A new live face check is required before allowing the user to cast a vote.

This helps in mitigating:

- Session hijacking
- Fraudulent voting
- Some forms of deepfake attacks

7.7.5 Security Measures for Face Verification

- Face encodings are securely stored and not directly exposed through APIs.
- High-accuracy thresholds are set to prevent false positives during facial verification.

7.8 Full Voting Process (With Layered Protection)

7.8.1 Vote Flow

The following steps are enforced during the voting process to ensure layered security:

1. Validate the session token.
2. Check whether the user has already voted by querying the `votes` table.
3. Prompt the user to upload a live face photo.
4. Perform AI-based facial verification.
5. If the face is successfully verified:
 - Hash the vote details.
 - Record the vote on the local custom blockchain.
 - Record the vote on the Ethereum blockchain through the deployed smart contract.
6. Log the voting event into the `vote_logs` table as:
 - Success
 - Duplicate vote attempt
 - Fraud attempt
 - System error

7.8.2 Duplicate Voting Protection

- The system ensures that a user cannot cast multiple votes.
- Every voting attempt, successful or failed, is logged for audit trail and traceability.

7.9 Role-Based Access Control (RBAC)

7.9.1 RBAC Integration

Session tokens contain encoded user roles such as:

- **Admin**
- **Official**
- **Voter**

7.9.2 Role Privileges

- Only officials are allowed to:
 - View vote counts
 - Access backend system statistics
 - Manage system configurations and operations
- Voters have restricted access limited to casting votes and viewing personal voting status.

7.9.3 API Access Verification

- Every API checks both the session token and the associated user role before proceeding.
- Non-authorized users are rejected with appropriate HTTP responses:
 - 401 Unauthorized
 - 403 Forbidden

7.10 Error Handling, Monitoring, and Debugging

7.10.1 Error Handling

- Extensive `try-except` blocks are used to catch and manage exceptions.
- Database transactions implement `commit/rollback` to ensure atomicity and prevent data corruption.

7.10.2 Debugging and Monitoring

- During development, stack traces and detailed errors are printed to the console.
- In production environments, errors can be redirected into logging systems for continuous monitoring.

7.10.3 User Feedback

- Error responses include meaningful messages sent back to the frontend for improved user experience and troubleshooting.

This system applies Zero Trust principles aggressively across all layers:

- Each user, each device, and each request is authenticated.
- MFA (OTP), session verification, and facial recognition ensure security is not just password-based.
- Blockchain (Ethereum + Local) provides immutability and transparency of votes.
- AI-powered face recognition provides an additional layer of biometric security atop digital authentications.
- All user activity is logged and auditable for traceability and forensic analysis.

Chapter 8

Tests results and Evaluation

8.1 Send OTP Test

8.1.1 Objective:

To ensure the One-Time Password (OTP) delivery system works reliably and securely as part of the Multi-Factor Authentication (MFA) mechanism.

8.1.2 Description:

In this test, the system simulates or triggers the action of sending an OTP to a registered user (via email or mobile). This process is crucial because OTP serves as a secondary layer of identity verification after password input but before face/biometric authentication.

8.1.3 Checks Performed:

- Is the OTP sent successfully to the correct user channel?
- Does the OTP expire after a certain time (e.g., 5 minutes)?
- Is the OTP unique and securely generated using cryptographic methods?
- Can the OTP be intercepted or reused?

8.1.4 Security Relevance:

This test ensures confidentiality and freshness of the authentication token, preventing impersonation and replay attacks.

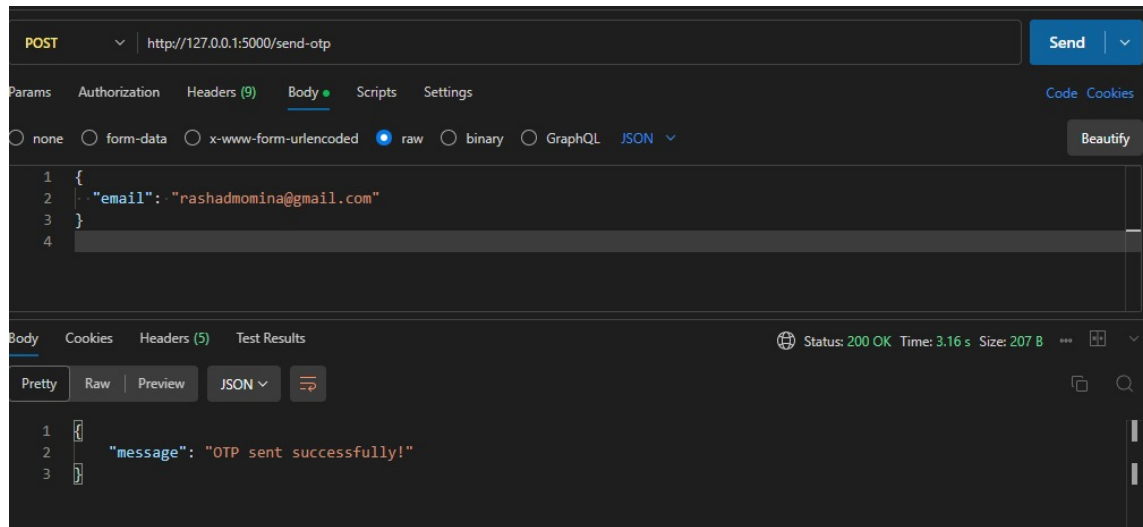


Figure 8.1: Testing Send OTP

8.2 Verify OTP Test

8.2.1 Objective:

To verify that only valid and unexpired OTPs are accepted and processed for user authentication.

8.2.2 Description:

After receiving the OTP, the user inputs it during login. This test validates:

- Whether the system accurately matches the OTP against stored (hashed) values.
- Whether it blocks expired or previously used OTPs.
- Whether incorrect or malformed OTPs are rejected with appropriate error messages.

8.2.3 Checks Performed:

- Validate OTP against user-session securely.
- Reject invalid or brute-forced OTPs.
- Logging invalid attempts for anomaly detection.

8.2.4 Security Relevance:

Protects against OTP brute-force attacks and ensures the integrity of the second authentication factor.

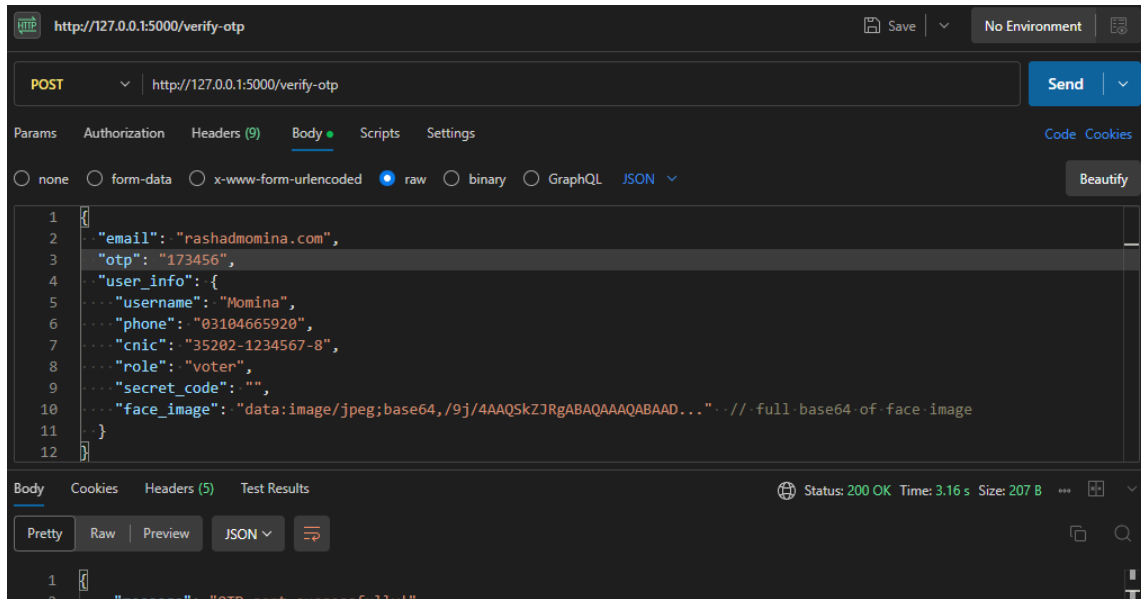


Figure 8.2: Verify OTP

8.3 Login Testing

8.3.1 Objective:

To validate the full login workflow under Zero Trust principles, ensuring that MFA, session handling, and identity validation function securely.

8.3.2 Description:

This test covers the end-to-end login sequence:

- Password input validation.
- OTP challenge.
- Biometric verification using FaceNet.
- Zero Trust policy checks like device ID, location, and session trustworthiness.

Checks Performed:

- Input sanitation (prevent SQLi, XSS, etc.).
- MFA layer flow integrity.

- AI-based verification (FaceNet accuracy & liveness).
- Unauthorized login attempts are blocked and logged.

8.3.3 Security Relevance:

This ensures Zero Trust posture by treating every login as untrusted by default, and verifying across multiple independent factors.

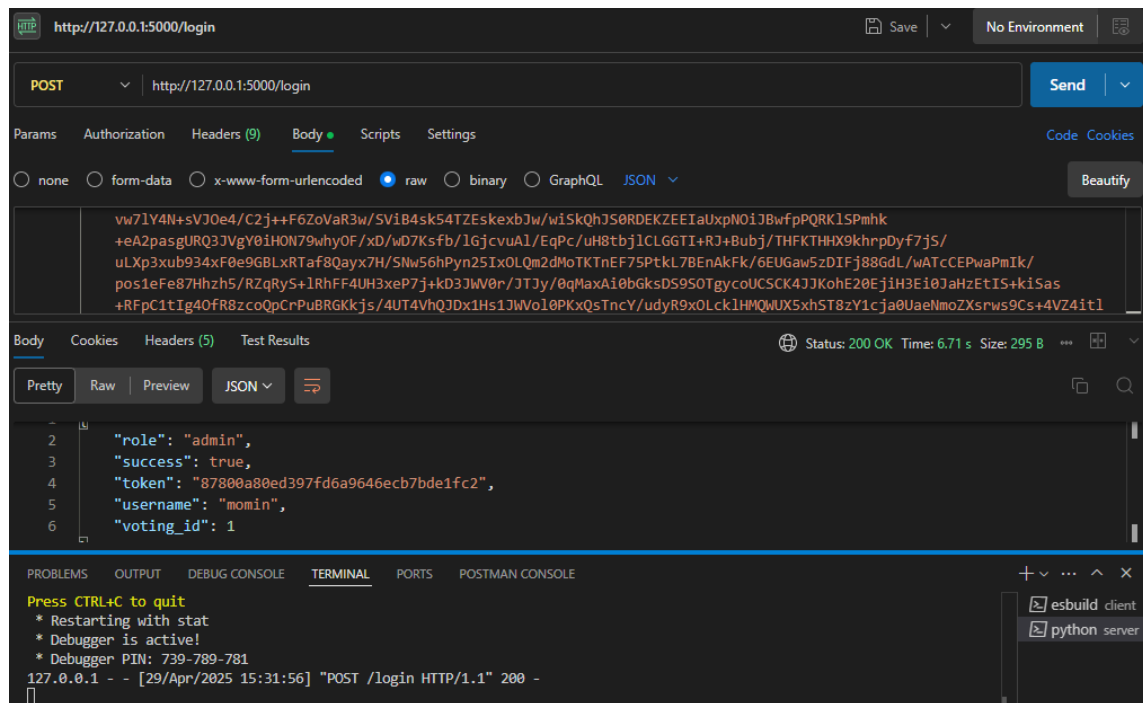


Figure 8.3: Login Testing

8.4 Protected Routes Testing

8.4.1 Objective:

To confirm that only authenticated and authorized users can access restricted parts of the application such as vote casting, admin panels, or vote verification.

8.4.2 Description:

Routes like `/vote`, `/admin/dashboard`, and `/voter/info` are guarded with authentication and Role-Based Access Control (RBAC). This test simulates:

- Authorized access by valid users.
- Unauthorized access attempts (e.g., token missing, token tampered).
- Access by wrong role types (e.g., voter trying to access admin route).

8.4.3 Checks Performed:

- JWT token validation and expiration.
- Role validation (admin, voter, observer).
- Secure error messaging (no leakage of internal logic).

8.4.4 Security Relevance:

Ensures authorization enforcement, session isolation, and Zero Trust route access policies

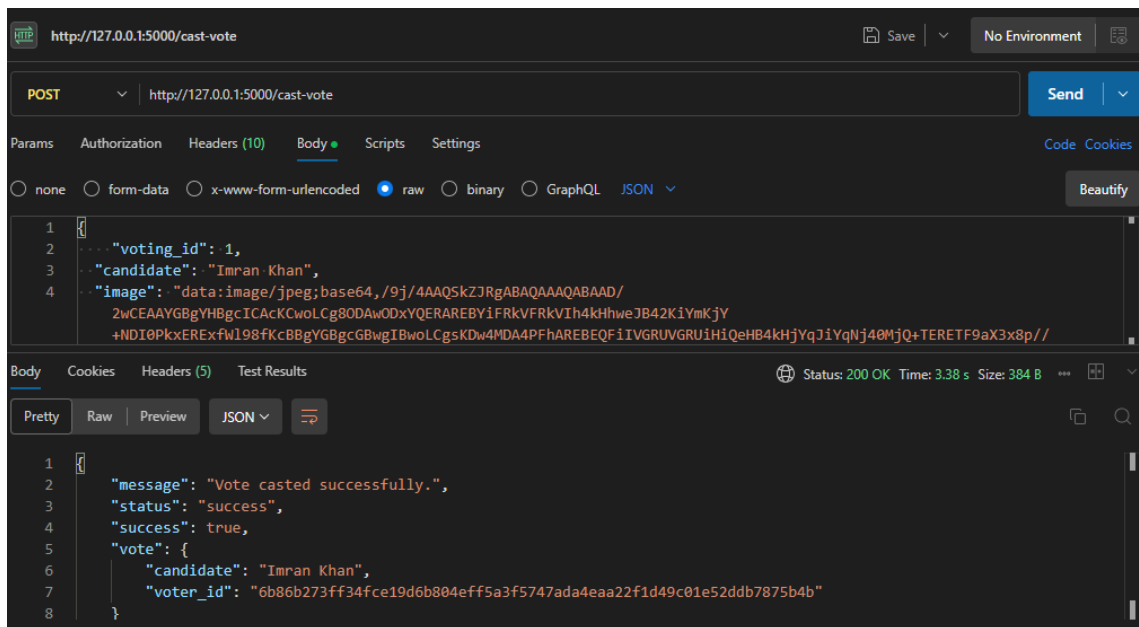


Figure 8.4: Protected Routes Testing

Chapter 9

Frontend Overview

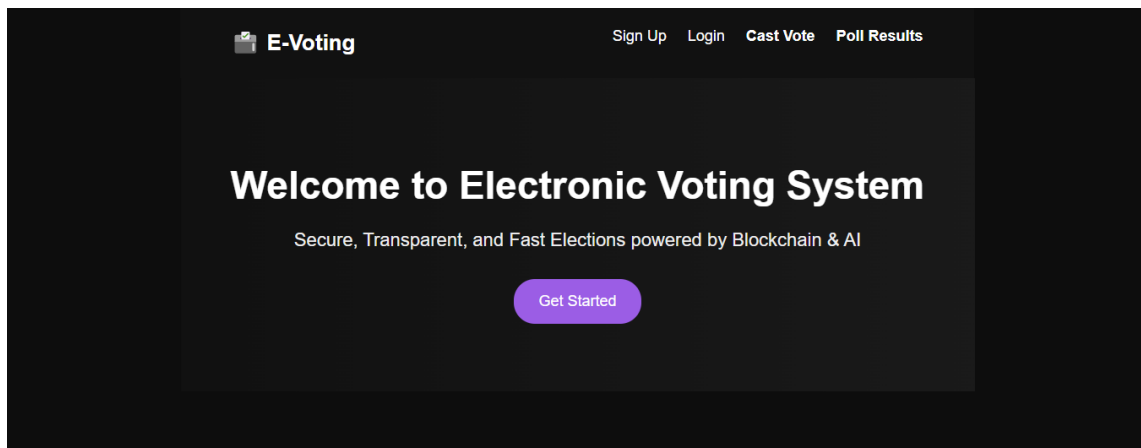


Figure 9.1: FrontPage Overview

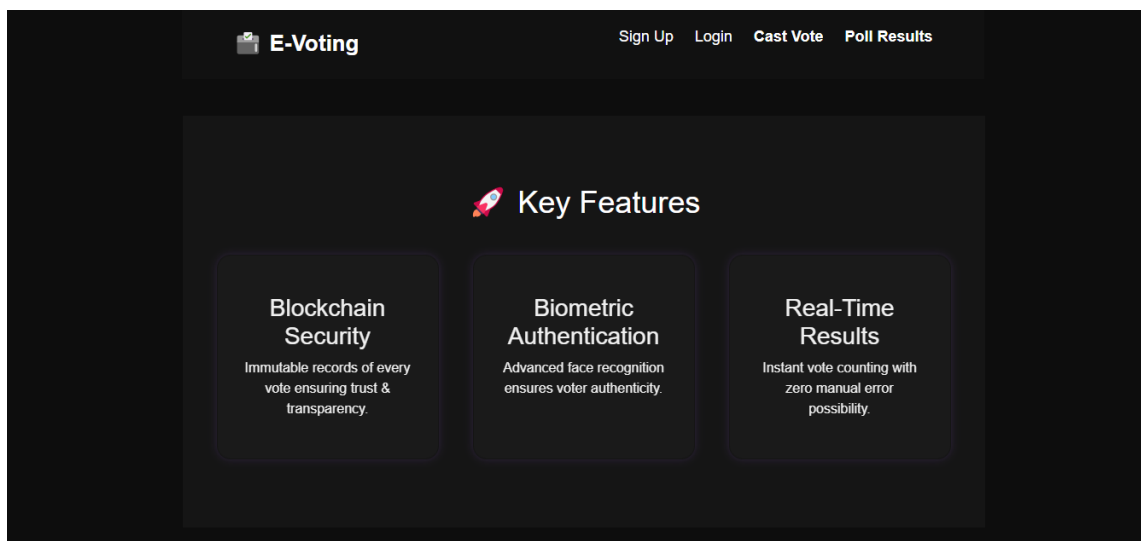


Figure 9.2: FrontPage Overview

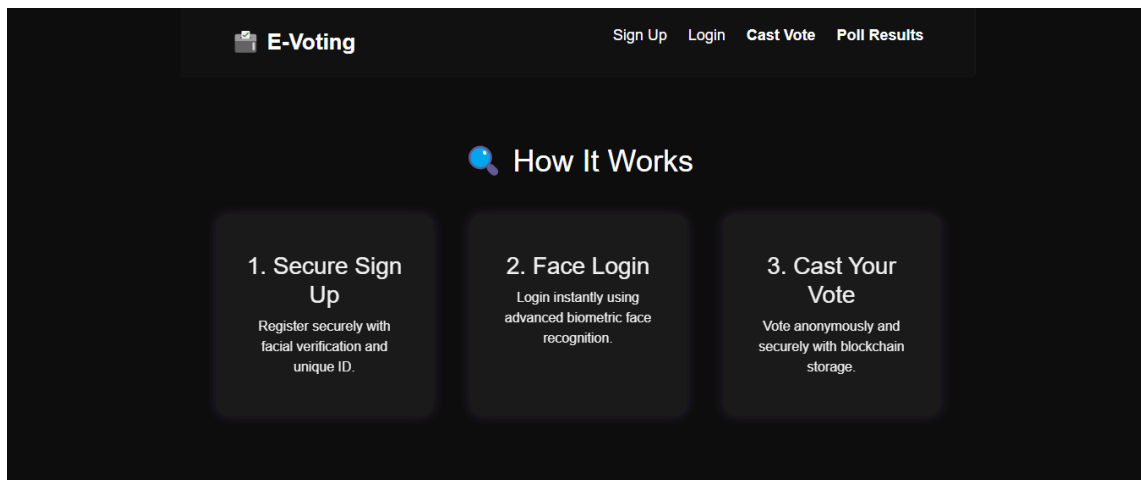


Figure 9.3: FrontPage Overview

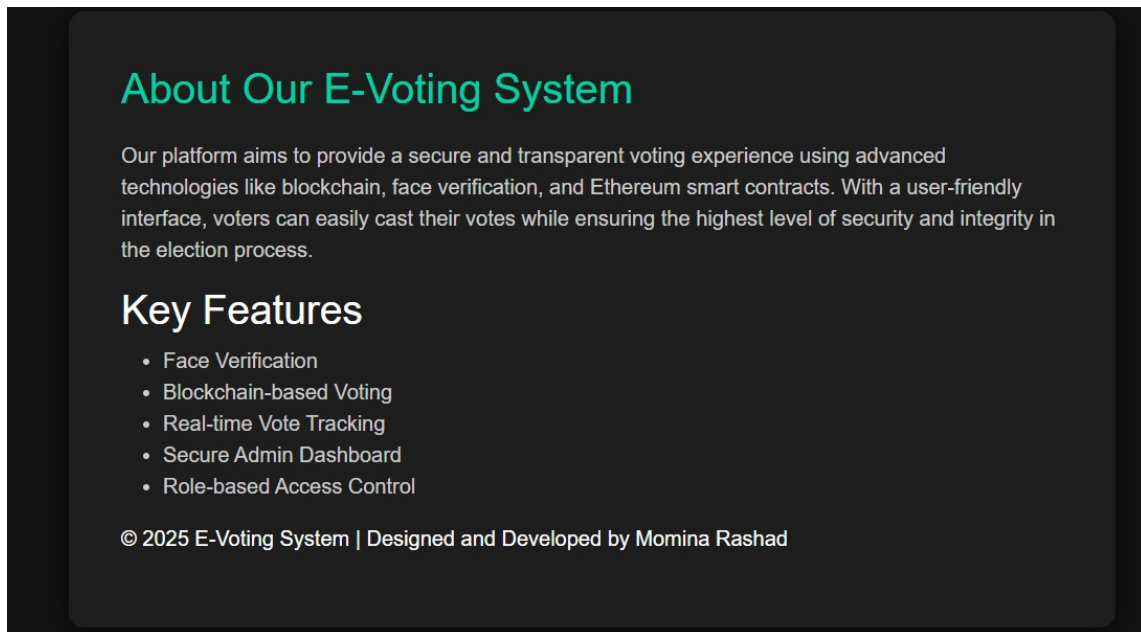
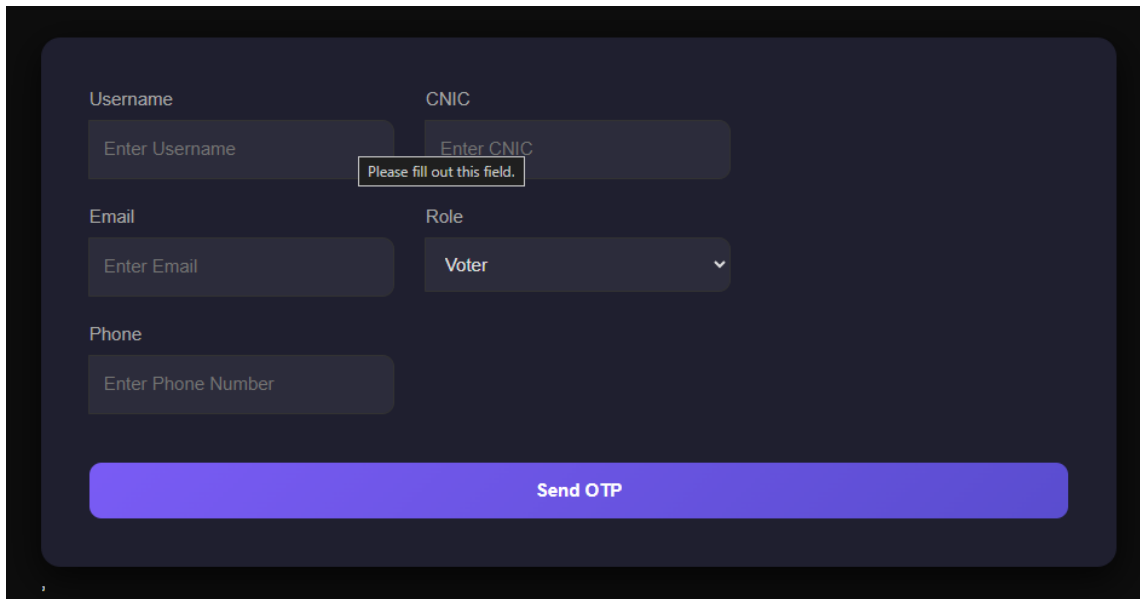
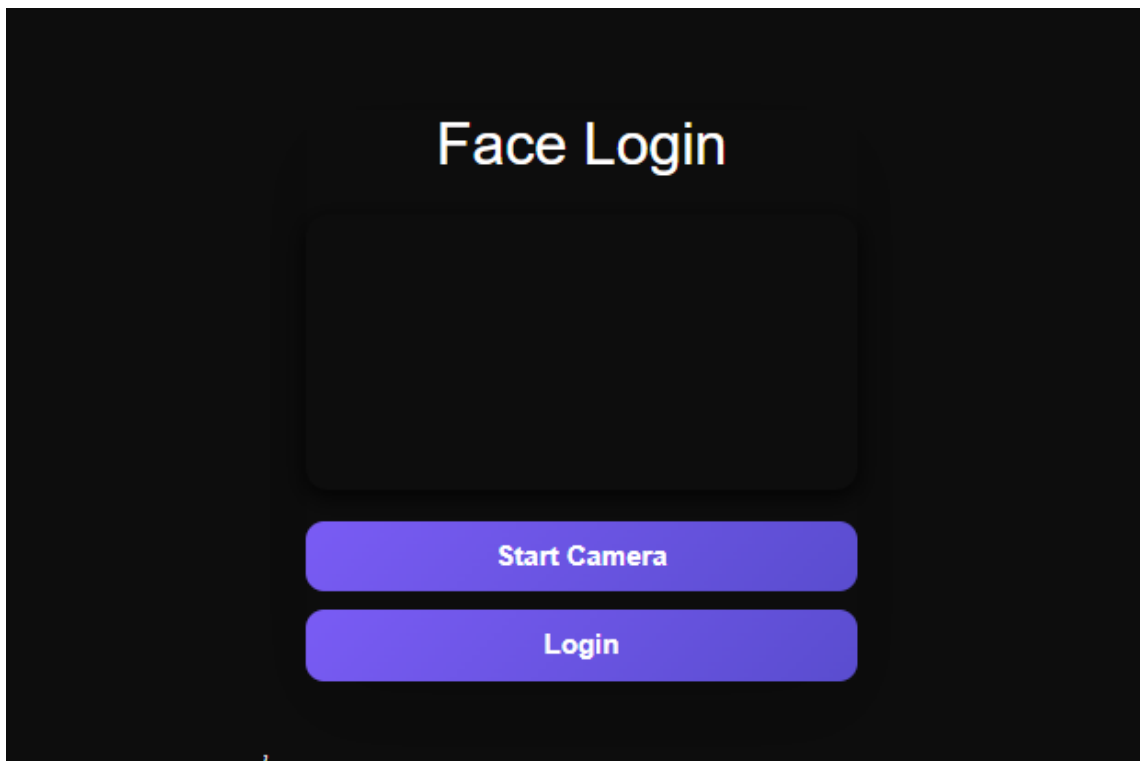


Figure 9.4: About



A dark-themed user registration form. It features five input fields: 'Username' (placeholder: 'Enter Username'), 'CNIC' (placeholder: 'Enter CNIC'), 'Email' (placeholder: 'Enter Email'), 'Phone' (placeholder: 'Enter Phone Number'), and 'Role' (a dropdown menu with 'Voter' selected). A small tooltip 'Please fill out this field.' is visible over the CNIC field. A large blue button labeled 'Send OTP' is positioned at the bottom of the form.

Figure 9.5: SignUp Form



A dark-themed login interface titled 'Face Login'. It features a large rectangular area for a face scan. Below this area are two blue buttons: 'Start Camera' and 'Login'.

Figure 9.6: Login Form

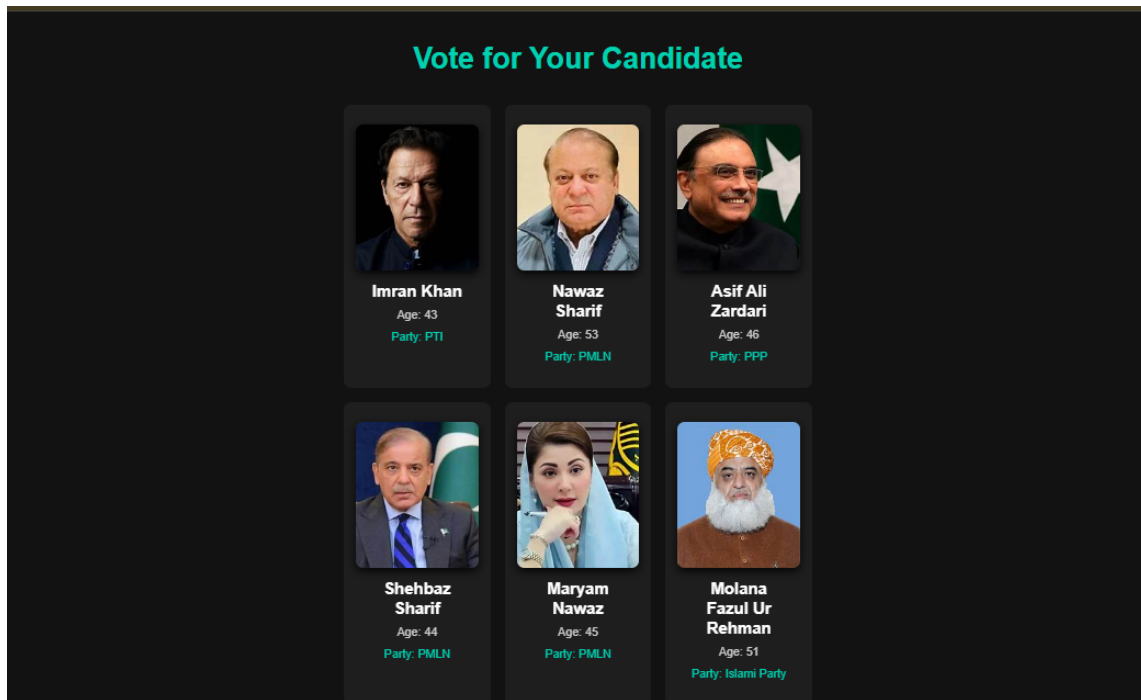


Figure 9.7: Candidates to Vote Page Overview



Figure 9.8: RBAC Page Overview

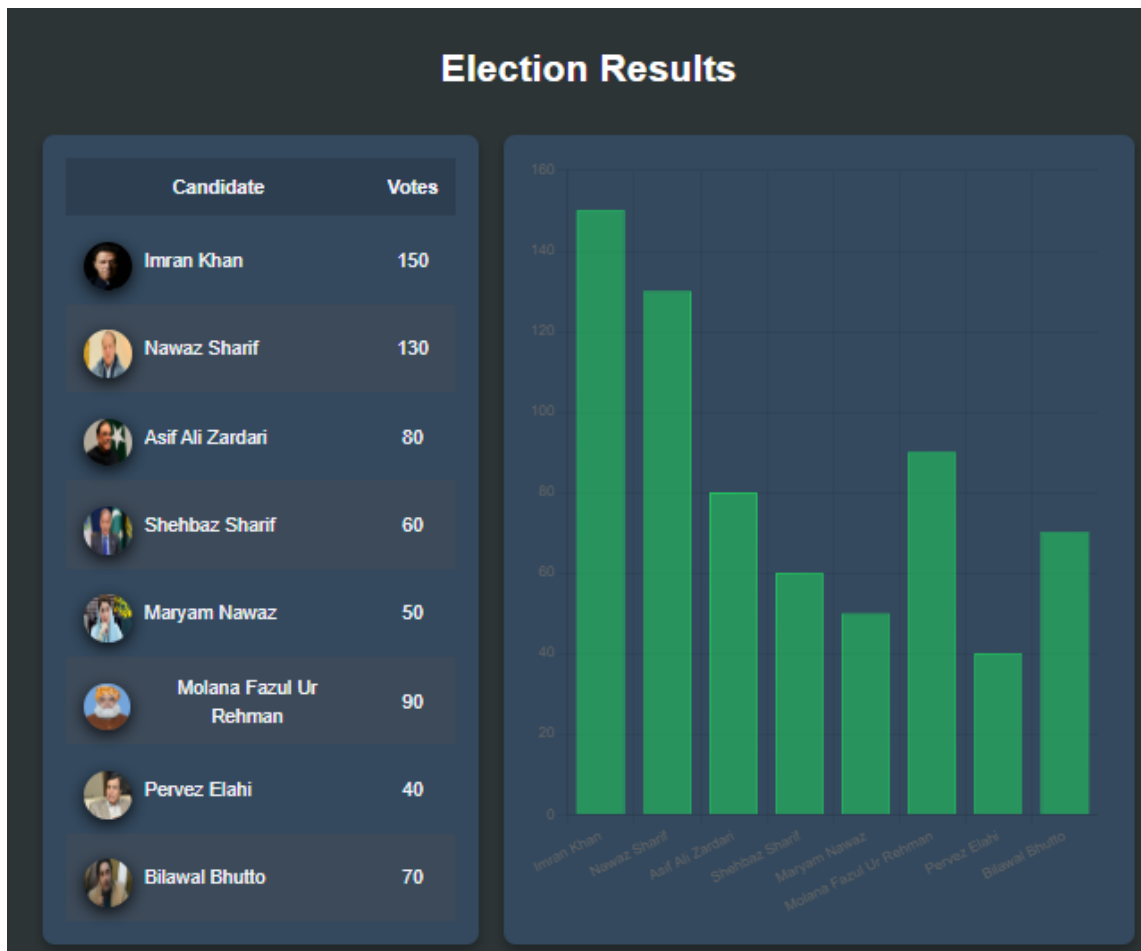


Figure 9.9: Election Results

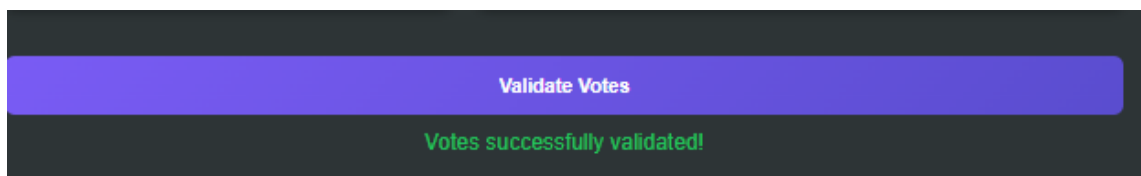


Figure 9.10: Validating Blockchain

Vote Integrity Logs					
Voting ID	IP Address	Candidate	Timestamp	Status	Reason
VOT1234	192.168.1.10	Imran Khan	4/29/2025, 2:40:18 PM	Fair Vote	-
VOT5678	192.168.1.20	Nawaz Sharif	4/29/2025, 2:40:18 PM	Possible Fraud	Multiple votes detected
VOT1122	192.168.1.30	Maryam Nawaz	4/29/2025, 2:40:18 PM	Fair Vote	-
VOT3344	192.168.1.40	Asif Ali Zardari	4/29/2025, 2:40:18 PM	Possible Fraud	Suspicious location activity
VOT5566	192.168.1.50	Bilawal Bhutto	4/29/2025, 2:40:18 PM	Fair Vote	-

Figure 9.11: Fraud Detection Page Overview

Contact Us

Name:

Email:

Subject:

Message:

Submit

Figure 9.12: Contact Form

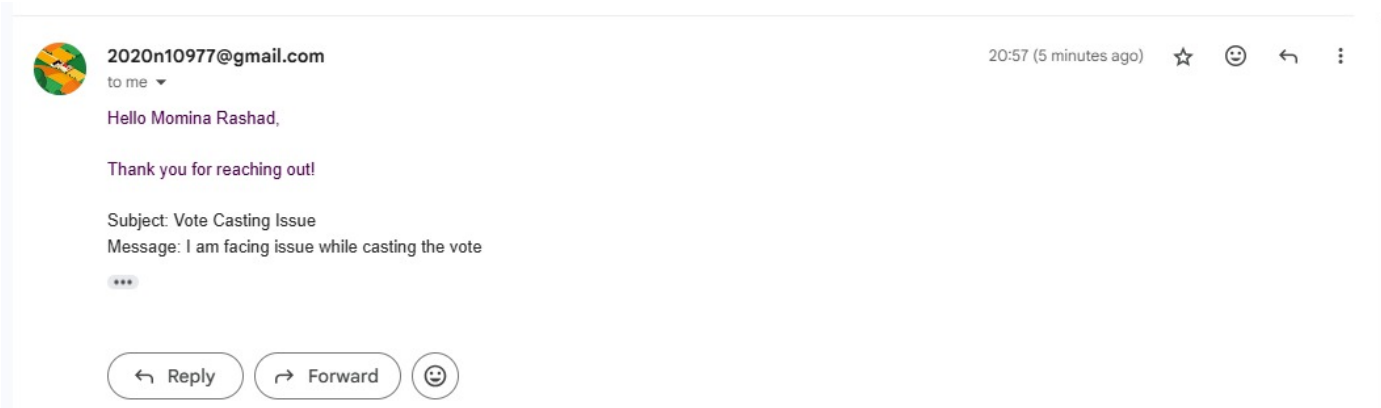


Figure 9.13: Mail received

Vote Integrity Logs					
Voting ID	IP Address	Candidate	Timestamp	Status	Reason
VOT1234	192.168.1.10	Imran Khan	4/29/2025, 2:40:18 PM	Fair Vote	-
VOT5678	192.168.1.20	Nawaz Sharif	4/29/2025, 2:40:18 PM	Possible Fraud	Multiple votes detected
VOT1122	192.168.1.30	Maryam Nawaz	4/29/2025, 2:40:18 PM	Fair Vote	-
VOT3344	192.168.1.40	Asif Ali Zardari	4/29/2025, 2:40:18 PM	Possible Fraud	Suspicious location activity
VOT5566	192.168.1.50	Bilawal Bhutto	4/29/2025, 2:40:18 PM	Fair Vote	-

Figure 9.14: Fraud Detection Page Overview

Chapter 10

Database Overview

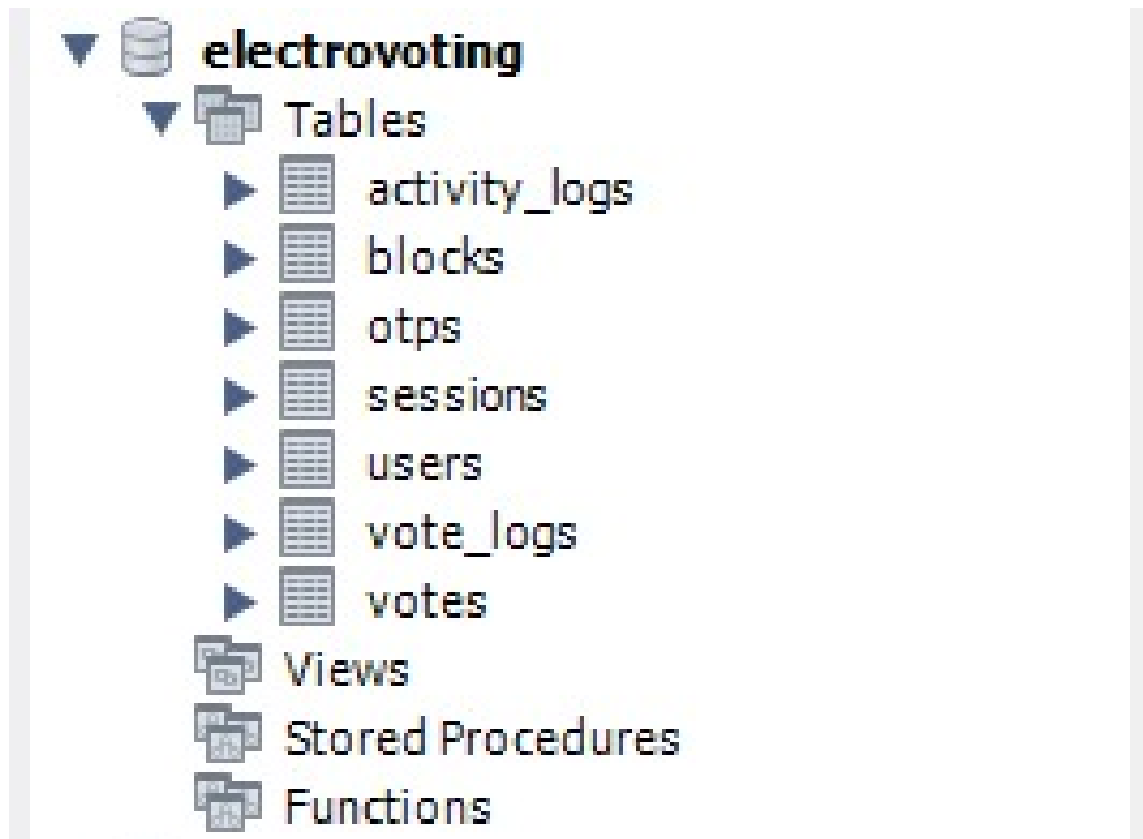


Figure 10.1: Database Tables

Result Grid								Filter Rows:	Edit:	Export/Import:	Wrap Cell Content:
	id	voting_id	role	token	created_at	expires_at	ip_address				
▶	1	1	official	6d1b297ef4f0325dda79bd84c7a1b4f4	2025-04-28 13:37:45	2025-04-28 13:42:45	127.0.0.1				
	2	2	voter	073849c5a7cf27d38d28869f00db6f32	2025-04-28 13:41:43	2025-04-28 13:47:23	127.0.0.1				
	3	1	official	51227b521999a4adfcc496fac1c73ebf	2025-04-28 14:32:16	2025-04-28 14:37:16	127.0.0.1				
	4	1	official	8f9faee0599a4391b379f35b3b5084a8	2025-04-28 14:49:35	2025-04-28 14:54:35	127.0.0.1				
	5	1	official	2075e3e923c91e4783d2223ea391237d	2025-04-28 15:08:28	2025-04-28 15:13:28	127.0.0.1				
	6	1	official	64dd75c5254111fa027dd6d317fd0532	2025-04-28 15:55:28	2025-04-28 16:00:28	127.0.0.1				
	7	1	official	9345bee1939ab506f3ec7872d5afe17a	2025-04-28 16:12:20	2025-04-28 16:17:20	127.0.0.1				
	8	1	official	824992fb8d396c67f5efdbf0e3c094c7	2025-04-28 16:22:47	2025-04-28 16:27:47	127.0.0.1				
	9	1	official	47a5ace229680c916ce59ed7ed63eec4	2025-04-28 16:34:13	2025-04-28 16:39:14	127.0.0.1				
	10	1	official	427b19a8a8ec662e5a1620a35706ec34	2025-04-28 16:58:41	2025-04-28 17:09:12	127.0.0.1				
	11	1	official	4fa4fbd082e6bb9d8713314665aaa8e8	2025-04-29 11:49:06	2025-04-29 11:55:07	127.0.0.1				
	12	1	official	03efb9b135b790f9fb4bd8ea88bc097f	2025-04-29 12:02:57	2025-04-29 12:07:57	127.0.0.1				
	13	1	official	7026d6f9a526f51fef0a87611cc4a37	2025-04-29 12:14:51	2025-04-29 12:19:51	127.0.0.1				

Figure 10.2: Session Management

Result Grid								Filter Rows:	Edit:	Export/Import:	Wrap C
	id	voting_id	ip_address	candidate	timestamp	status	reason				
▶	1	2	127.0.0.1	Imran Khan	2025-04-28 13:41:49	success	Unread result found				
	2	2	127.0.0.1	Pervez Elahi	2025-04-28 13:42:23	error	Unread result found				
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL				

Figure 10.3: Authenticated users are restricted to a single vote

	id	voting_id	username	email	phone	cnic	role	is_verified	face_encoding
	1	1	moi	rashadmomina@gmail.com	0912...	352...	official	1	AAAA4DtIwb8AAABgVBysPwAAAKA0tqA/AAAA
▶	2	2	gugu	mominarashad137@gmail.com	0312...	365...	voter	1	AAAAAGacwr8AAADAskmkPwAAAMCyo5s/AAA
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Figure 10.4: Encoding of Facial Data

Chapter 11

Recommendations for Real-World Deployment

11.1 Secure Handling of Sensitive Information

In the present setup, sensitive credentials like database passwords, Gmail SMTP credentials, and blockchain private keys are directly included in the codebase.

Recommendation

- Move all sensitive settings to environment variables or secure encrypted vaults.
- Utilize Python libraries such as `python-dotenv` to securely load sensitive keys at runtime.
- Enforce access control measures to allow only authorized processes or users access to sensitive environment variables.
- Periodically rotate secrets to reduce the effect of leaks.

11.2 Enhance Session Management and Authentication

The system uses randomly generated session tokens stored in a database. While this is a good start, it can be enhanced.

Recommendation

- Replace simple session tokens with JWT (JSON Web Tokens) signed by a private server-side key.
- Use brief session timeout (e.g., 5–10 minutes) with refresh tokens.

- Use IP pinning and device fingerprinting methods to identify irregular login activities.
- Forcefully invalidate sessions if any suspicious behavior or multiple logins at the same time from different IPs are noticed.

11.3 Enhance Email OTP Delivery Mechanism

Currently, the application sends OTPs through Gmail SMTP using a personal account, which can lead to rate limiting, unreliability, and spam flags.

Recommendation

- Use a transactional email service such as SendGrid, Mailgun, or Amazon SES for reliable delivery.
- Implement OTP request rate-limiting to prevent brute-force bot attacks.
- Store the timestamp of the last OTP and block successive OTP sends within a short timeframe (e.g., 60 seconds).

11.4 Enhance Face Recognition Security

Currently, facial recognition technology relies on fundamental facial encodings that utilize the `face_recognition` library.

Recommendation

- Upgrade to more secure alternatives like FaceNet, Dlib CNN, or DeepFace for feature extraction.
- Implement liveness detection techniques, including blink detection, head movement detection, or anti-spoofing systems.
- Assess adversarial attack threats (e.g., printed photo attacks) and secure the recognition system against them.

11.5 Streamline Blockchain Interaction

Votes are currently persisted directly to a local Ethereum testnet (Ganache).

Recommendation

- For production, use public Ethereum Testnets (e.g., Goerli) first for final testing, and then mainnet or private networks (e.g., Quorum, Hyperledger Besu).
- Instead of storing every vote transaction individually, group and hash batches of votes to reduce gas fees and network congestion.
- Develop a strategy for smart contract improvement that supports the inclusion of future additions or features.

11.6 Hardening MySQL Database

Database utilization is efficient but may experience bottlenecks at high load.

Recommendation

- Create indexes on fields like `voting_id`, `email`, and `voter_hash` to speed up lookups.
- Enhance query performance using query caching and connection pooling.
- Regularly back up databases and encrypt backups.
- Apply strict input validation and use prepared statements (already partially implemented) to prevent SQL injection.

11.7 Best Practices for Smart Contract Security

The use of smart contracts with inadequate auditing can result in severe vulnerabilities.

Recommendation

- Have the Solidity code professionally audited.
- Employ security libraries such as Mythril, Slither, and OpenZeppelin libraries to write secure contracts.
- Use access control with modifiers such as `onlyOwner`, and protect critical functions such as vote tallying or chain verification.

11.8 Sophisticated Voting Fraud Detection

Face duplication checks at the time of casting votes are excellent but can be improved.

Recommendation

- Examine IP addresses and user behavior for irregular patterns (e.g., excessive voting from one network).
- Utilize risk scoring — if circumstances are questionable, require more verification steps.
- Create a complete fraud investigation dashboard for monitoring by the admin.

11.9 System Logging and Error Monitoring

Errors are presently returned via JSON responses but are not actively logged.

Recommendation

- Use centralized logging by employing platforms like Sentry, Logstash, or CloudWatch.
- Monitor and alert on anomalies like excessive error rates, high latencies, or database connectivity errors.
- Maintain a failure recovery plan, e.g., database recovery, session recovery, and blockchain state recovery.

11.10 Production-Grade Server Deployment

The Flask application is using the built-in development server, which is not intended for production.

Recommendations

- Deploy Flask using either uWSGI or Gunicorn, behind an Nginx proxy server for high availability.
- Activate HTTPS (SSL/TLS) to encrypt communication and prevent man-in-the-middle attacks.
- Utilize Docker containers to facilitate easy deployment and scalability across different environments.

11.11 Authentication Enhancements

The existing authentication system relies exclusively on the recognition of facial features.

Recommendation

- Implement multi-factor authentication (MFA) — i.e., OTP upon successful face recognition login.
- Enable users to configure trusted devices for quicker logins but authenticate new devices more aggressively.
- Implement role-based access control (RBAC) for administrators, officials, and users.

11.12 Legal Compliance and Data Privacy

The management of biometric information, including facial encodings, presents various legal liabilities.

Recommendation

- Comply with privacy regulations such as GDPR, CCPA, and PDPA depending on where you are based.
- Maintain an active data protection policy detailing why you are collecting biometric data, how you are using it, and when it will be deleted.
- Offer users an option to request data deletion or opt-out of biometric data collection if required.
- Encrypt facial information both in transit and at rest.

11.13 Frontend Enhancements and User Experience

Currently, backend APIs support crucial operations effectively, yet the frontend experience is not fully addressed in the code.

Recommendation

- Improve user feedback after actions like login, OTP verification, or voting with better UI/UX design.
- Enforce automatic session expiration notifications on the frontend.
- Provide real-time feedback for vote status, blockchain verification, and election results where feasible.

Chapter 12

Conclusion

This project guarantees resilience against cyberattacks, integrity, and transparency by providing a complete security framework addressing the crucial concerns with digital voting systems. Combining blockchain-based vote storage, artificial intelligence-driven fraud detection, and Zero Trust Security ideas helps to eliminate vulnerabilities including illegal access, duplicate voting, vote manipulation, and data breaches.

12.1 Key Takeaways from the Proposed Solution

12.1.1 Zero Trust Security Implementation:

Every user and system interaction undergoes continuous authentication and strict access control, preventing unauthorized access at every stage of the voting process.

12.1.2 AI-Powered Fraud Detection:

Machine learning models proactively identify anomalies, duplicate votes, and suspicious behavior, ensuring that fraudulent activities are detected in real-time.

12.1.3 Blockchain-Based Vote Storage:

The immutable, decentralized ledger ensures that votes cannot be altered, deleted, or manipulated, fostering trust and transparency in election results.

12.1.4 Enhanced Election Security:

Multi-Factor Authentication (MFA), Just-in-Time (JIT) vote access, and AI-based anomaly monitoring create a highly secure voting environment resistant to cyber threats.

12.2 Impact and Future Potential

This research sets a new standard for secure digital elections, proving that advanced security frameworks can successfully protect democratic processes from cyberattacks and election fraud. Beyond elections, the proposed Zero Trust + AI + Blockchain model can be adapted for:

12.2.1 Secure Identity Verification

Secure identity verification in government and private sectors.

12.2.2 Fraud-Resistant Financial Transactions

Fraud-resistant financial transactions using AI anomaly detection.

12.2.3 Tamper-Proof Record-Keeping

Tamper-proof record-keeping systems across industries.

By leveraging the latest advancements in cybersecurity and artificial intelligence, this project ensures the future of digital voting is secure, reliable, and resistant to manipulation, strengthening public confidence in online elections worldwide.

Bibliography

- [1] A. Suresh, A. Gupthan, S. Abhishek *et al.*, “Secure vote: Ai-powered fingerprint authentication for next-generation online voting,” in *2023 7th International Conference on Electronics, Communication and Aerospace Technology (ICECA)*. IEEE, 2023, pp. 993–1000.
- [2] N. R. Dyavani and M. Thanjaivadivel, “Advanced security strategies for cloud-based e-commerce: Integrating encryption, biometrics, blockchain, and zero trust for transaction protection.”
- [3] A. Sivamuganthan, L. M. Dass, S. R. Mahalingam, P. L. Hang, K. Kelvin, J. Wingsky, A. M. M. A. Mohamed, A. J. Rebecca, N. I. Y. B. A. Hatmal, A. R. Ali *et al.*, “The role of cryptography and ai in enhancing blockchain security,” 2025.
- [4] S. Ahmed, K. Wongmahesak, B. Singh, and S. Kumar, “Empowering democratic processes with ai: Innovations in voter engagement, policy analysis, and decision-making process,” in *Democracy and Democratization in the Age of AI*. IGI Global Scientific Publishing, 2025, pp. 121–132.
- [5] N. I. of Standards and T. (NIST), “Zero trust architecture (sp 800-207),” NIST, Tech. Rep., 2020. [Online]. Available: <https://csrc.nist.gov/publications/detail/sp/800-207/final>
- [6] I. Goodfellow, J. Shlens, and C. Szegedy, “Explaining and harnessing adversarial examples,” *arXiv preprint*, 2015. [Online]. Available: <https://arxiv.org/abs/1412.6572>
- [7] N. Papernot, P. McDaniel, and I. Goodfellow, “Practical black-box attacks against machine learning,” in *Proceedings of ACM on Computer and Communications Security (CCS)*, 2017. [Online]. Available: <https://arxiv.org/abs/1602.02697>
- [8] A. Kurakin, I. Goodfellow, and S. Bengio, “Adversarial machine learning at scale,” *ICLR Conference Proceedings*, 2018. [Online]. Available: <https://arxiv.org/abs/1611.01236>
- [9] M. Corporation, “Mitre att&ck for ai security,” 2023. [Online]. Available: <https://attack.mitre.org/>

- [10] I. O. for Standardization (ISO), *ISO/IEC 27001:2022 Information Security Management Systems*, Std., 2022. [Online]. Available: <https://www.iso.org/standard/54534.html>
- [11] V. Poornachandran, “Securing the vote: Leveraging anomaly detection in blockchain systems,” *International Journal of Engineering Research and Sustainable Technologies (IJERST)*, vol. 2, no. 4, pp. 19–25, 2024.
- [12] I. Security, “Zero trust for ai and cloud security: Enhancing digital resilience,” 2022. [Online]. Available: <https://www.ibm.com/security/zero-trust>
- [13] M. Security, “Applying zero trust principles to ai workflows and cloud-based systems,” 2023. [Online]. Available: <https://www.microsoft.com/security/blog/>
- [14] M. B. E. Islam, M. Haseeb, H. Batool, N. Ahtasham, and Z. Muhammad, “Ai threats to politics, elections, and democracy: a blockchain-based deepfake authenticity verification framework,” *Blockchains*, vol. 2, no. 4, pp. 458–481, 2024.
- [15] H. Choung, P. David, and T.-W. Ling, “Acceptance of ai-powered technology in surveillance scenarios: Role of trust, security, and privacy perceptions,” *Security, and Privacy Perceptions*.